

In the last few lectures, we talked about class of  $P$ .

These are problems that you can avoid brute forcing exponential search space.

However, There are some problems that we cannot avoid brute force search space.

One important discovery about these type of problems shows that complexities of many problems are related.

## Definition

A verifier for a Language  $A$  is an algorithm  $V$ , where

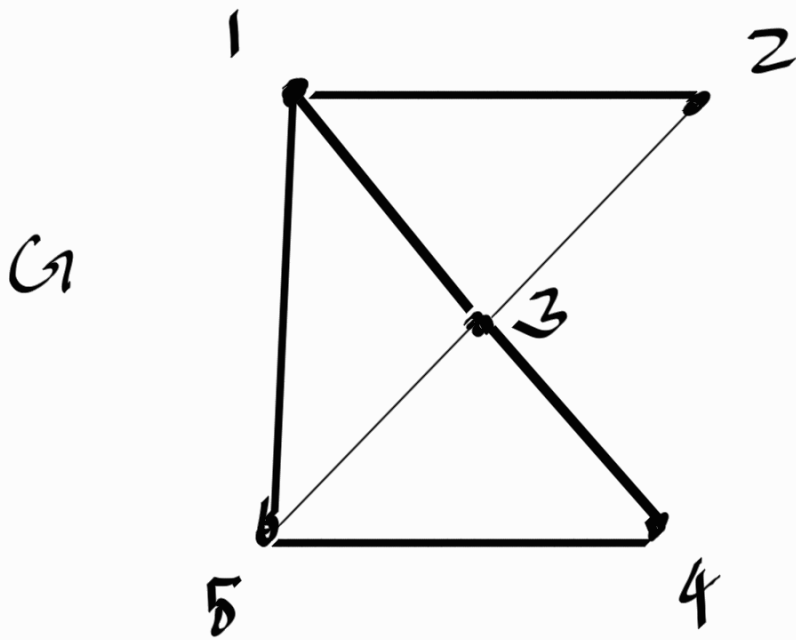
$$A = \{w \mid V \text{ accepts } \langle w, c \rangle \text{ for some string } c\}$$

We measure the time of the verifier in terms of the length of the  $w$ , so a polynomial time verifier runs in polynomial time in the length of  $w$ . A language  $A$  is polynomially verifiable if it has polynomial time verifier.

$c$  - some additional information to help  $V$  to verify that  $w$  is a member of the language  $A$ .

# Hamiltonian Cycle

Given a graph  $G=(V,E)$ , is there a cycle that visits every vertex exactly once and return to the original vertex?



is there a  
HC in  $G$

$\langle 1, 2, 3, 4, 5, 1 \rangle$

$\langle 3, 4, 5, 1, 2, 3 \rangle$

$\langle 2, 1, 3, 4, 5, 1, 2 \rangle$

for this problem we can come up with a polynomial time verifier.

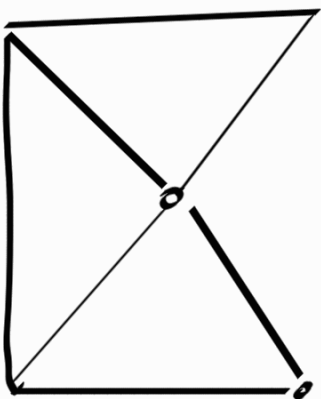
Solving the  
problem

$\neq$

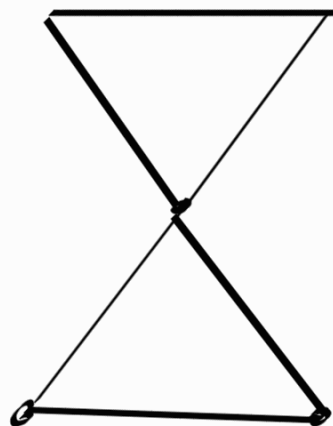
verifying the  
problem

A language  $A$  is polynomially verifiable if it has a verifier that runs in polynomial time in the length of the input  $w$ .

$\overline{HC} = \{G \mid G \text{ has no hamiltonian cycle}\}$



$G_1$



$G_2$

for  $\overline{HC}$ , it is unknown whether we can have a polynomial verifier.

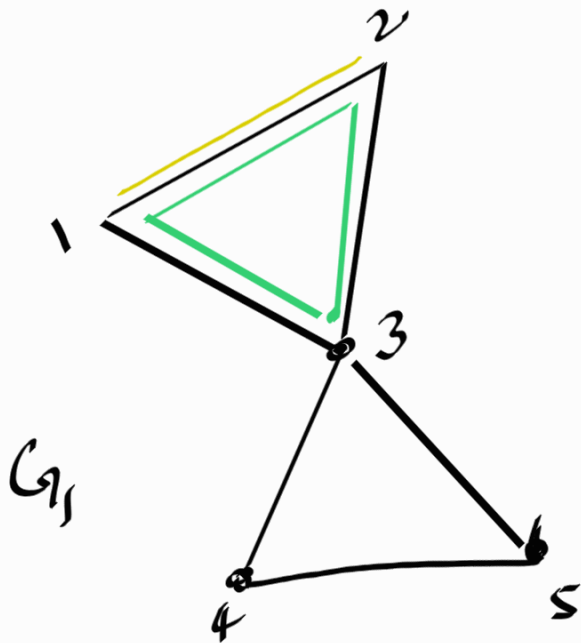


## Definition

NP is the class of languages that has <sup>a</sup>polynomial time verifiers.

The term NP comes from  
Nondeterministically polynomial time.

$K\text{-CLIQUE} = \{ \langle G, k \rangle \mid G \text{ is an undirected graph of } n \text{ vertices which contains a } k\text{-clique or } k\text{-complete-subgraph} \}$



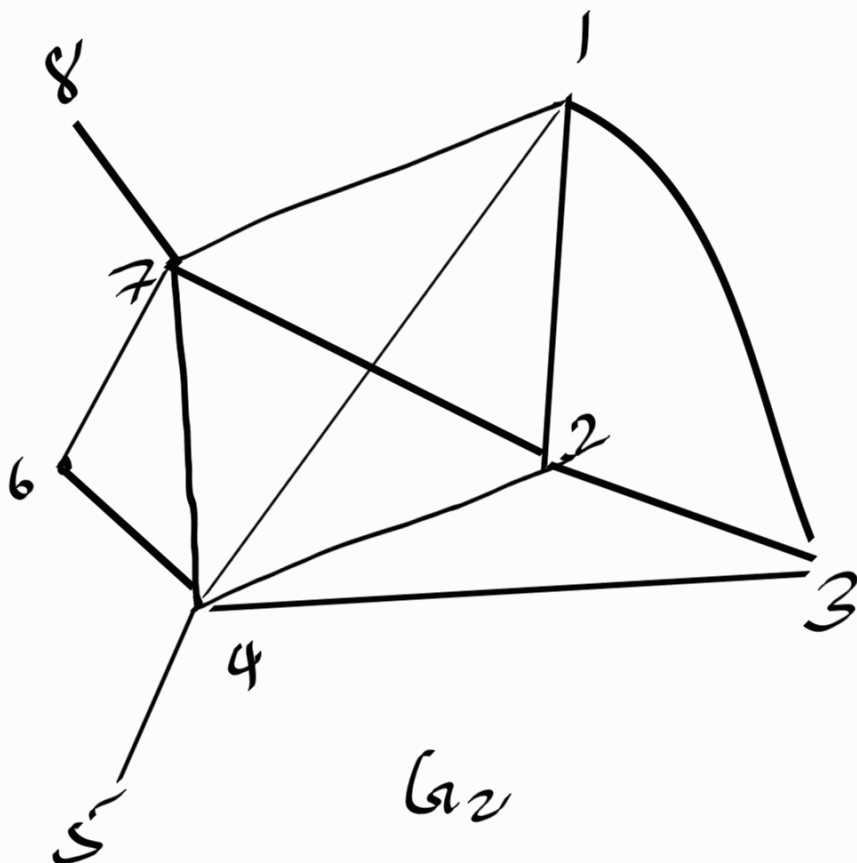
2-clique

3-clique



$G$  has a 2-clique

$G$  has a 3-clique.



$\{1, 2, 4, 7\}$

Let us define a polynomial time verifier for CLIQUE

$A = 1$  on input  $\langle \langle G, k \rangle, c \rangle$

1. Test whether  $c$  is a set of  $k$  nodes in  $G$ .
2. Test if  $G$  contains all  $\binom{k}{2}$  edges containing nodes in  $c$ .
3. If both passes, accept otherwise, reject.

Runtime:  $O(nk + \binom{k}{2})$

There is a different definition for NP.

### Theorem 7.20

A language is NP iff it is decided by some nondeterministic polynomial time Turing machine.

Def of nondeterministic TM.

In an NTM, at any point in a computation, machine may proceed according to several possibilities.

$$\delta: Q \times \Gamma \longrightarrow P(Q \times \Gamma \times \{L, R\})$$

### Theorem 3.16

Every NTM has an equivalent TM.

- refer Theorem 3.16 for the proof

## Proof idea for 7.20

We show that we can convert a polytime verifier to an equivalent polynomial time NTM and vice versa,

NTM simulates the verifier by guessing all possible certificates. The verifier simulates the NTM by using the accepting branch as the certificate.

Let  $L$  be a language.

$L \in NP \iff$  Some NTM  $N$  decides  $L$ .

$\Rightarrow$

Let  $V$  be the verifier for  $L$

$N =$  "on input  $w$  of length  $n$ :"

1. Nondeterministically select string  $c$  of length at most  $n^k$
2. Run  $V$  on input  $\langle w, c \rangle$
3. If  $V$  accepts, accept, otherwise, reject

⇐

let  $L$  be decided by NTM  $N$

we construct the poly time verifier as follows

$V =$  "on input  $\langle w, c \rangle$ , where  $w, c$  are strings.

1. Simulate  $N$  on  $w$ , treating each symbol of  $c$  as a description of the nondeterministic choice to make at each step
2. If this branch of  $N$ 's computation accepts, accept; otherwise, reject.

## Side Note:

Think of a NTM as a TM that can compute multiple choices simultaneously.

## Analogy

Model

Real-world analogy

TM

Following recipes one by one.

NTM

Instantly checking all possible recipes and picking the right one.



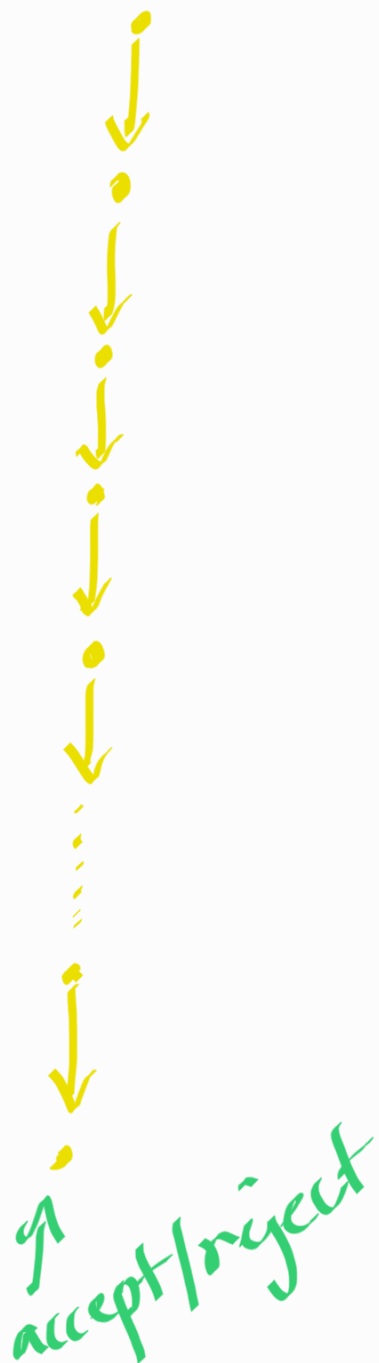
NTM can be deciders as well.

## Definition

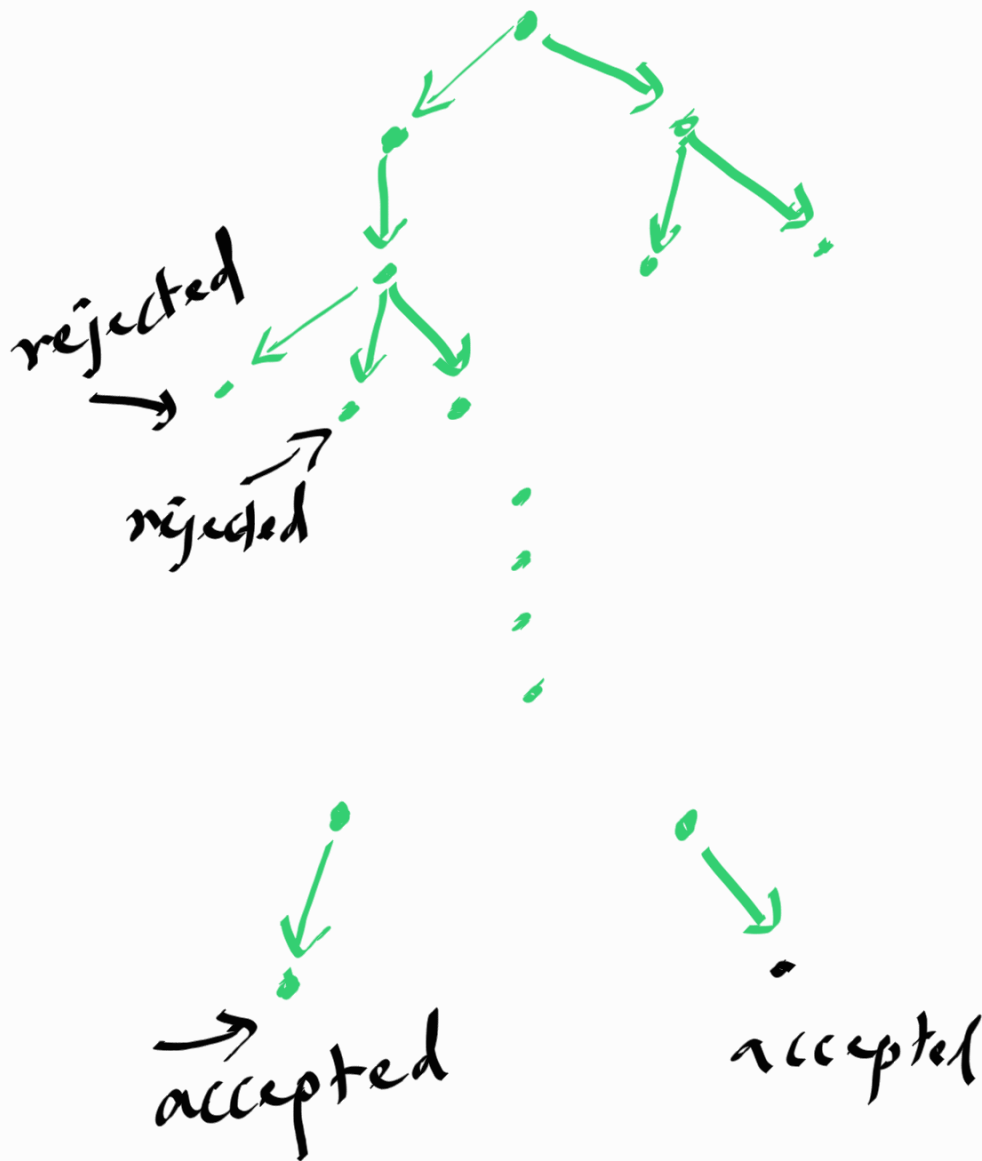
Let  $N$  be a nondeterministic turing machine that is a decider. The running time of  $N$  is the function  $f: \mathbb{N} \rightarrow \mathbb{N}$ , where  $f(n)$  is the maximum number of steps that  $N$  uses on any branch of its computation on any input of length  $n$ .

If a NTM is a decider, then every computation path must halt.

Deterministic  
TM for problem  $A$ ,  
 $M$



Non deterministic  
TM  $N_A$  for problem



## Example: CLIQUE $\in$ NP

suppose NTM  $A$  decides CLIQUE

$A = 1$  on input  $\langle G, k \rangle$ ,  $G$  is a undirected graph,  $k \leq n = \#$  of vertices in the graph.

1. Nondeterministically select a set  $C$  of  $k$  nodes from  $G$ .
2. Test if  $G$  contains all  $\binom{k}{2}$  edges connecting nodes in  $C$ .
3. If the test passes, accept. otherwise, reject.

$N$  runs in  $O\left(\binom{k}{2}\right) = O(n^2)$

CLIQUE  $\in$  NP

$NTIME(t(n)) = \{L \mid L \text{ is a language decided by } O(t(n)) \text{ time nondeterministic TM}\},$

$- NP = \bigcup_K NTIME(n^K), K \text{ is a constant}$

# Recap

$P$  = the class of languages for which membership can be decided quickly.

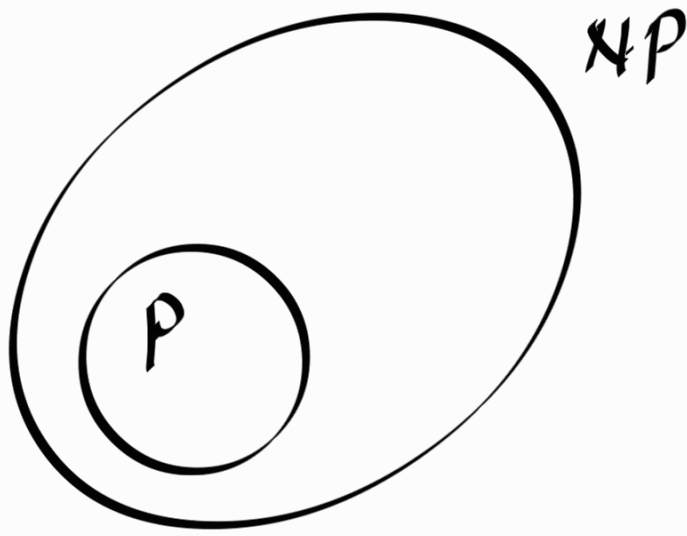
$NP$  = the class of languages for which membership can be verified quickly.

Languages like CLIQUE are in  $NP$ , but we don't know whether they are in  $P$ .

$$P \stackrel{?}{=} NP$$

$$P \subseteq NP \checkmark$$

$$P \stackrel{?}{\supseteq} NP$$



In NP and P, we only have decision problems.

- No optimization problems.

Note:

A language  $L$  is in NP if there exist

- A deterministic poly-time algorithm  $V(x, w)$  called verifier
- A polynomial  $p(n)$

s.t

$$x \in L \iff \exists w : |w| \leq p(|x|) \wedge V(x, w) = 1$$